



南開大學
Nankai University

计算机学院
机器学习实验报告

实验 5: 聚类分析

姓名：王旭

学号：2312166

专业：计算机科学与技术

2025 年 12 月 10 日

目录

1 实验名称	3
2 实验目的	3
3 层次聚类 (Hierarchical Clustering) 基本原理	3
3.1 层次聚类概述	3
3.2 层次聚类的工作流程	3
3.3 Single-linkage (单链接) 原理	3
3.4 Complete-linkage (全链接) 原理	4
3.5 两种方法的对比分析	4
3.5.1 相似之处:	4
3.5.2 差异之处:	4
4 基础任务: Single-linkage 与 Complete-linkage 聚类	4
4.1 实验要求	4
4.2 代码实现:	5
4.2.1 Single-linkage 策略	5
4.2.2 Complete-linkage 策略	6
4.3 运行结果与分析	7
4.3.1 原始数据分布图 original_data.png	7
4.3.2 Single-linkage 聚类与 Complete-linkage 聚类结果分布图	8
4.3.3 单链接 Single-linkage 与多链接 Complete-linkage 结果的对比分析	9
5 中级要求: Average-linkage 聚类	10
5.1 实验要求	10
5.2 Average-linkage (平均链接) 聚类理论原理	10
5.3 代码实现	11
5.4 运行结果与分析	11
5.4.1 Average-Linkage 聚类结果分布图	12
5.4.2 从理论方面解释 Average-linkage 表现最好的原因	12
6 提高要求: 算法对比与结论	13
6.1 实验要求	13
6.2 运行结果与分析	13
6.2.1 对噪声/离群点的敏感度分析	14
6.2.2 簇的形状偏好分析	14
6.2.3 可视化结果对比分析	15
7 拓展任务: 变换聚类簇数 k 分析	16
7.1 运行结果概览	16
7.2 不同 k 值下的图表分析	17
7.2.1 ARI vs k 图分析	17

7.2.2	Silhouette vs k 图分析	17
7.3	k 值对聚类性能的影响分析	18
7.3.1	k=2 (小于真实簇数)	18
7.3.2	k=3 (等于真实簇数)	18
7.3.3	k=4-6 (大于真实簇数)	18
8	github 仓库	18

1 实验名称

聚类分析

2 实验目的

1. 理解并掌握层次聚类 (Hierarchical Clustering) 的基本原理;
2. 实现并对比 Single-linkage (单链接)、Complete-linkage (全链接) 和 Average-linkage (平均链接) 三种不同距离度量方式的聚类效果;
3. 利用可视化手段展示聚类前后的样本分布, 并通过调整聚类簇数 (k 值) 分析算法性能。

3 层次聚类 (Hierarchical Clustering) 基本原理

3.1 层次聚类概述

层次聚类是一种通过计算样本之间的相似度来构建层次化聚类结构的算法。主要分为两种:

- **凝聚式 (Agglomerative)**: 自底向上, 每个样本开始是一个簇, 然后逐步合并
- **分裂式 (Divisive)**: 自顶向下, 所有样本开始是一个簇, 然后逐步分裂

我们实验中使用的是**凝聚式层次聚类**。

3.2 层次聚类的工作流程

1. 开始: 将每个样本点视为一个单独的簇
2. 计算所有簇之间的距离矩阵
3. 找到距离最近的两个簇, 将它们合并为一个新簇
4. 更新距离矩阵, 反映新簇与其他簇的距离
5. 重复步骤 3-4, 直到所有样本合并为一个簇 (或达到预设的簇数 k)

3.3 Single-linkage (单链接) 原理

数学定义:

两个簇 C_i 和 C_j 之间的距离定义为:

$$D_{sl}(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y)$$

即两个簇之间的距离定义为两个簇中最近样本点之间的距离。

算法特点：

- ”最近邻”策略：只要两个簇中有一对点距离很近，就会合并
- 链式效应 (Chaining Effect)：容易形成长链状的聚类结构
- 对噪声敏感：离群点可能将本应分离的簇连接起来
- 适合发现非球形簇：能够识别任意形状的聚类

3.4 Complete-linkage（全链接）原理**数学定义：**

两个簇 C_i 和 C_j 之间的距离定义为：

$$D_{cl}(C_i, C_j) = \max_{x \in C_i, y \in C_j} d(x, y)$$

即两个簇之间的距离定义为两个簇中最远样本点之间的距离。

算法特点：

- ”最远邻”策略：要求两个簇中所有点对的最大距离都要小
- 产生紧凑簇：倾向于形成球形的、紧密的聚类
- 对噪声敏感：离群点可能阻止本应合并的簇合并
- 适合发现球形簇：对分离良好的球形簇效果最好

3.5 两种方法的对比分析**3.5.1 相似之处：**

- 都是层次聚类算法
- 都需要构建树状图 (dendrogram)
- 都需要预设聚类数量 k 或设置距离阈值

3.5.2 差异之处：

特征	Single-linkage	Complete-linkage
距离定义	最近点对距离 (min)	最远点对距离 (max)
簇形状	链状、不规则	紧凑、球形
对噪声敏感度	高 (易受离群点影响)	高 (易受离群点影响)
适用场景	非球形、任意形状簇	分离良好的球形簇

4 基础任务：Single-linkage 与 Complete-linkage 聚类**4.1 实验要求****1. Single-linkage 实现与绘图：**

- 使用 Single-linkage 策略对生成的数据进行聚类;
- 设置聚类簇数 k (建议与生成数据时的 `centers` 一致, 如 $k=3$);
- 绘制对比图: 左图为原始未分类分布, 右图为聚类后的结果 (不同簇用不同颜色标记)。

2. Complete-linkage 实现与绘图:

- 使用 Complete-linkage 策略重复上述步骤;
- 同样绘制聚类结果分布图。

4.2 代码实现:

4.2.1 Single-linkage 策略

我们知道, 对于 Single-linkage 策略而言, 两个簇 C_i 和 C_j 之间的距离定义为:

$$D_{sl}(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y)$$

即两个簇之间的距离定义为两个簇中最近样本点之间的距离。因此我们从该原理出发, 完成代码中 `single_linkage_distance` 部分的函数实现。

单链接策略 `single_linkage_distance` 函数实现

```

1  def _single_linkage_distance(self, cluster1, cluster2, dist_matrix):
2      """
3      Single-linkage: 两个簇之间的【最小】距离
4      输入:
5          cluster1: 第一个簇的样本索引列表, 如 [0, 3, 5]
6          cluster2: 第二个簇的样本索引列表, 如 [1, 2]
7          dist_matrix: 距离矩阵, shape=(n_samples, n_samples)
8      输出:
9          两个簇之间的最小距离
10     """
11     min_dist = np.inf # 初始化为无穷大, 确保第一次比较能更新
12
13     # 遍历 cluster1 中的每个样本
14     for i in cluster1:
15         # 遍历 cluster2 中的每个样本
16         for j in cluster2:
17             # 从预计算的距离矩阵中获取距离, 即获取当前样本对的距离
18             current_dist = dist_matrix[i, j]
19             # 如果当前距离更小, 更新最小值
20             if current_dist < min_dist:
21                 min_dist = current_dist
22     return min_dist

```

代码实现的流程:

- 初始化最小距离 `min_dist` 为无穷大
- 双重循环遍历两个簇 `cluster1` 与 `cluster2` 的所有样本对 (i, j)

- 使用 `dist_matrix[i, j]` 获取 i 和 j 的距离, 比较并更新最小距离 (`current_dist` 与 `min_dist`)
- 返回找到的最小距离 `min_dist`

4.2.2 Complete-linkage 策略

我们知道, 对于 Complete-linkage 策略而言, 两个簇 C_i 和 C_j 之间的距离定义为:

$$D_{cl}(C_i, C_j) = \max_{x \in C_i, y \in C_j} d(x, y)$$

即两个簇之间的距离定义为两个簇中最远样本点之间的距离。因此我们从该原理出发, 完成代码中 `complete_linkage_distance` 部分的函数实现。

全链接策略 `complete_linkage_distance` 函数实现

```

1  def _complete_linkage_distance(self, cluster1, cluster2, dist_matrix):
2      """
3      Complete-linkage: 两个簇之间的【最大】距离
4      输入:
5          cluster1: 第一个簇的样本索引列表
6          cluster2: 第二个簇的样本索引列表
7          dist_matrix: 距离矩阵
8      输出:
9          两个簇之间的最大距离
10     """
11     max_dist = 0 # 初始化为0, 确保第一次比较能更新
12
13     # 遍历 cluster1 中的每个样本
14     for i in cluster1:
15         # 遍历 cluster2 中的每个样本
16         for j in cluster2:
17             # 获取当前样本对的距离
18             current_dist = dist_matrix[i, j]
19             # 如果当前距离更大, 更新最大值
20             if current_dist > max_dist:
21                 max_dist = current_dist
22
23     return max_dist

```

代码实现的流程:

- 初始化最大距离 `max_dist` 为无穷大
- 双重循环遍历两个簇 `cluster1` 与 `cluster2` 的所有样本对 (i, j)
- 使用 `dist_matrix[i, j]` 获取 i 和 j 的距离, 比较并更新最大距离 (`current_dist` 与 `max_dist`)
- 返回找到的最大距离 `max_dist`

4.3 运行结果与分析

在实现单链接 Single-linkage 与全链接 Complete-linkage 聚类后，我们运行整体的代码，观察实验结果。

```
PS D:\机器学习\实验五> python -u "d:\机器学习\实验五\exp_5_template.py"
实验五：层次聚类分析
=====
[步骤1] 生成人工数据集 (make_blobs)...
样本数: 150, 特征数: 2
真实类别数: 3
图片已保存: out_exp5\original_data.png

[步骤2] 基本要求: Single-linkage 和 Complete-linkage

Single-linkage聚类...
ARI: -0.0001
图片已保存: out_exp5\single_linkage.png

Complete-linkage聚类...
ARI: 0.7481
图片已保存: out_exp5\complete_linkage.png
```

图 4.1: 代码输出结果

下面我们从生成的聚类结果分布图入手，来进行详细地分析。

4.3.1 原始数据分布图 original_data.png

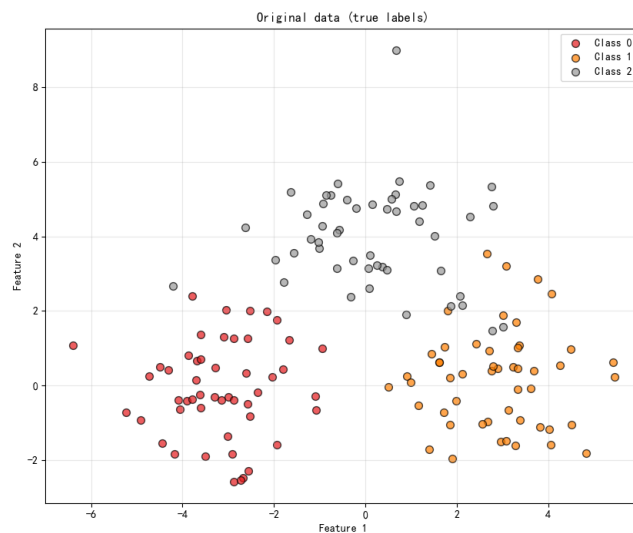


图 4.2: 原始数据分布图

我们对原始数据首先进行分析:

- **数据样本:** 150 个二维点
- **真实类别:** 3 个 (红色、黄色、灰色)
- **数据形态:** 3 个相对紧凑的球形簇, 三个簇呈近似球形分布, 且簇的大小和密度相对均匀
- **分布特点:** 簇间有一定间隔与边界, 但可能存在一些边界重叠点
- 这是 `make_blobs` 生成的典型高斯混合数据

4.3.2 Single-linkage 聚类与 Complete-linkage 聚类结果分布图

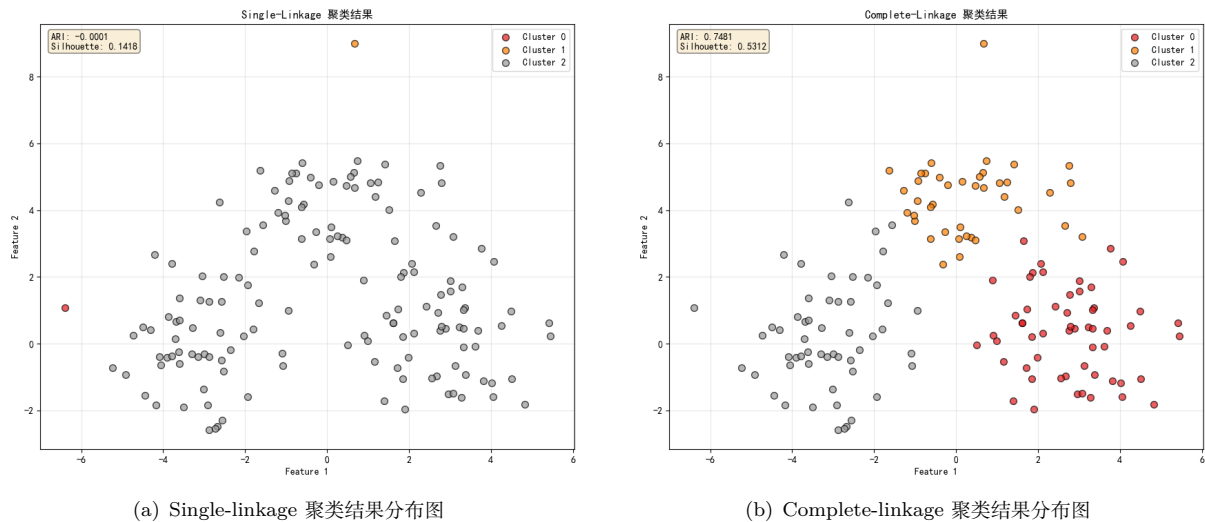


图 4.3: 两个策略下的聚类结果分布图

(1) Single-linkage 聚类结果分析

- **ARI = -0.0001**: 接近 0, 表示聚类结果与真实标签几乎没有相关性
- **Silhouette = 0.1418**: 接近 0, 表示样本聚类质量很差
- **视觉观察**: 大部分样本被归为 Cluster 2 (灰色), 只有少数几个离群点被单独划分为 Cluster 0 和 Cluster 1

(*) 从理论方面分析 Single-linkage 表现差的原因:

1. 链式效应 (Chaining Effect)

- Single-linkage 基于最小距离, 只要两个簇中有任何一对点距离很近就会合并
- 在球形数据中, 相邻簇的边缘点可能比同簇的中心点更接近
- 这导致簇像链条一样连接起来, 最终大部分点被合并到同一个大簇中

2. 对边界点敏感

- 原始数据中, 三个簇之间有一些边界点
- Single-linkage 会通过这些边界点将整个簇连接起来
- 形成了”一个主簇 + 多个孤立点”的结构

3. 数学原理体现

- Single-linkage: $D_{sl} = \min d(x, y)$
- 只要存在一对靠近的点, 距离就很短, 容易合并
- 导致算法过早地合并了本应分离的簇

(2) Complete-linkage 聚类结果分析

- **ARI = 0.7481**: 较高, 表示聚类结果与真实标签有较强的一致性
- **Silhouette = 0.5312**: 表明聚类结构良好
- **视觉观察**: 三个簇的划分清晰, 与原始数据分布高度吻合

(*) 从理论方面分析 Complete-linkage 表现好的原因:

1. 紧凑性要求

- Complete-linkage 基于最大距离, 要求两个簇中所有点对的距离都要小
- 这迫使算法只合并那些内部点都相互接近的紧凑簇
- 正好匹配球形簇的特征

2. 抵制链式效应

- 即使两个簇的最近点距离很小, 只要最远点距离大, 就不会轻易合并
- 有效防止了通过边缘点连接的链式效应
- 保持簇的独立性

3. 数学原理体现

- Complete-linkage: $D_{cl} = \max d(x, y)$
- 只有两个簇的整体距离都小时才会合并
- 对紧凑球形簇有天然的优势

4.3.3 单链接 Single-linkage 与多链接 Complete-linkage 结果的对比分析

维度	Single-linkage	Complete-linkage	分析
ARI	-0.0001	0.7481	Complete-linkage 显著更好
Silhouette	0.1418	0.5312	Complete-linkage 聚类质量更高
簇大小	极不平衡 (一个大簇 + 多个小簇)	相对平衡	Single-linkage 的链式效应导致
边界处理	敏感, 易通过边界点连接	稳健, 要求整体接近	关键差异点
适用性	适合链状、非球形簇	适合紧凑球形簇	本实验数据适合 Complete-linkage

表 1: Single-linkage 与 Complete-linkage 结果对比分析表

实验结论与思考

1. Complete-linkage 在球形数据上表现优异, ARI 达到 0.7481
2. Single-linkage 不适用于球形数据, 出现严重链式效应
3. 距离度量选择对聚类结果有决定性影响
4. 评价指标意义: ARI 和 Silhouette 都体现了 Complete-linkage 的优势

5 中级要求：Average-linkage 聚类

5.1 实验要求

1. 实现 Average-linkage 层次聚类算法；
2. 在同样的 k 值下，绘制样本分布图；
3. 观察该结果与前两种方法在边缘样本划分上的区别。

5.2 Average-linkage（平均链接）聚类理论原理

1. 数学定义与公式

Average-linkage 定义两个簇 C_i 和 C_j 之间的距离为：

$$D_{avg}(C_i, C_j) = \frac{1}{|C_i| \times |C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$$

其中：

- $|C_i|$ 和 $|C_j|$ 分别是两个簇的样本数量
- $d(x, y)$ 是样本点 x 和 y 之间的距离（欧氏距离）
- 求和符号表示计算所有可能的样本对距离之和

2. 算法思想核心

平衡策略：Average-linkage 在 Single-linkage 和 Complete-linkage 之间找到一个平衡点：

- 既不像 Single-linkage 那样只关注最近点（容易产生链式效应）
- 也不像 Complete-linkage 那样只关注最远点（容易产生过于紧凑的簇）
- 而是考虑**所有点对的平均距离**，更加全面和稳健

特征	Single-linkage	Complete-linkage	Average-linkage
距离定义	最小距离 (min)	最大距离 (max)	平均距离 (avg)
簇形状	链状、不规则	紧凑、球形	均衡、中等紧凑
对噪声敏感度	高（易受离群点影响）	高（易受离群点影响）	低（较为稳定）
计算复杂度	中等	中等	较高
适用场景	非球形、任意形状	球形、分离良好	通用、多种形状

表 2: 三种层次聚类方式在原理方面的对比

3. 边缘样本处理

Average-linkage 在边缘样本划分上的独特优势：

- 当两个簇有部分重叠时，Average-linkage 能更好地识别边界
- 对于处于簇边界的样本，归属判断更加合理
- 避免了 Single-linkage 的”过度连接”和 Complete-linkage 的”过度分离”

5.3 代码实现

Average-linkage 策略

我们知道，对于 Average-linkage 策略而言，两个簇 C_i 和 C_j 之间的距离定义为：

$$D_{avg}(C_i, C_j) = \frac{1}{|C_i| \times |C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$$

即两个簇之间的距离定义为两个簇中所有样本点对之间距离的平均值。因此我们从该原理出发，完成代码中 `average_linkage_distance` 部分的函数实现。

平均链接策略 average_linkage_distance 函数实现

```

1  def _average_linkage_distance(self, cluster1, cluster2, dist_matrix):
2      """
3      Average-linkage: 两个簇之间的【平均】距离
4      输入：
5          cluster1: 第一个簇的样本索引列表
6          cluster2: 第二个簇的样本索引列表
7          dist_matrix: 距离矩阵
8      输出：
9          两个簇之间的平均距离
10     """
11     total_dist = 0.0 # 总距离，初始化为0.0确保浮点运算
12     count = 0 # 样本对计数器
13
14     # 遍历第一个簇的所有样本
15     for i in cluster1:
16         # 遍历第二个簇的所有样本
17         for j in cluster2:
18             # 累加当前样本对的距离
19             total_dist += dist_matrix[i, j]
20             count += 1 # 增加计数
21
22     # 计算平均距离：总距离除以样本对数量
23     if count > 0:
24         return total_dist / count
25     else:
26         return 0.0 # 如果两个簇都是空的，返回0

```

代码实现的关键部分：

- 累加所有样本对的距离
- 除以样本对的数量 ($|C1| \times |C2|$)

5.4 运行结果与分析

在实现平均链接 Average-Linkage 策略聚类后，我们运行整体的代码，观察实验结果。

```
[步骤2] 基本要求: Single-linkage 和 Complete-linkage
```

```
Single-linkage聚类...
ARI: -0.0001
图片已保存: out_exp5\single_linkage.png
```

```
Complete-linkage聚类...
ARI: 0.7481
图片已保存: out_exp5\complete_linkage.png
```

```
[步骤3] 中级要求: Average-linkage
```

```
ARI: 0.8332
图片已保存: out_exp5\average_linkage.png
```

图 5.4: 代码输出结果

下面我们从生成的聚类结果分布图入手, 来进行详细地分析。

5.4.1 Average-Linkage 聚类结果分布图

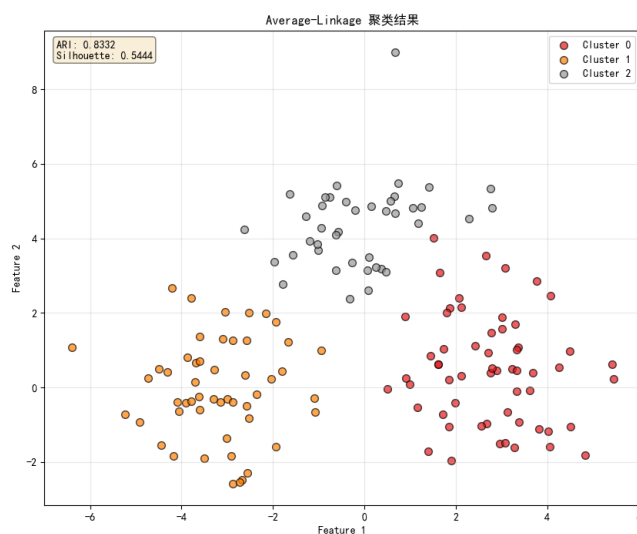


图 5.5: Average-linkage 聚类结果分布图

从图中观察到的特点:

1. 清晰的簇划分: 三个簇的边界非常清晰
2. 合理的边缘点分配: 边界点的划分看起来更加自然
3. 紧凑的簇结构: 每个簇内部相对紧凑
4. 与真实分布高度一致: 与原始数据图的分布几乎一致

我们发现:

Average-linkage 不仅如预期地介于 Single 和 Complete 之间, 反而**超过了 Complete-linkage 的表现**, ARI 达到了 0.8332, 是所有方法中最高的。

5.4.2 从理论方面解释 Average-linkage 表现最好的原因

(1) 数学原理的优势 Average-linkage 的公式:

$$D_{avg}(C_i, C_j) = \frac{1}{|C_i| \times |C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$$

优势体现：

1. **整体性考虑**：不仅考虑极端点，而是所有点的平均关系
2. **统计稳定性**：大数定律使得平均值对异常值不敏感
3. **自然平衡**：在紧凑性和连通性之间取得最佳平衡

(2) 在球形数据上的具体优势 对于 `make_blobs` 生成的球形数据：

1. **簇内同质性**：每个簇内部点距离相近
2. **簇间分离性**：不同簇的质心距离较远
3. **平均距离的优势**：能准确反映簇间的整体关系

(3) 三种方法在算法行为方面的差异

Single-linkage：只看最近点 → 过度连接 → 形成一个大簇

Complete-linkage：只看最远点 → 过度分离 → 可能过于保守

Average-linkage：看所有点 → 平衡决策 → 最符合实际

6 提高要求：算法对比与结论

6.1 实验要求

在实验报告中对比上述三种算法（Single, Complete, Average）在当前数据集上的表现。

1. 对噪声/离群点的敏感度：哪种算法容易受个别离群点影响？
2. 簇的形状：哪种算法更倾向于发现球形簇？哪种适合不规则形状？

6.2 运行结果与分析

```
[步骤4] 提高要求：算法对比 (make_blobs 数据集)

[算法对比]
-----
linkage      ARI      silhouette
-----
single       -0.0001    0.1418
complete     0.7481    0.5312
average      0.8332    0.5444
-----

结论 (make_blobs 数据集):
- 在当前数据集上, average-linkage 表现最好
- Single-linkage: 容易产生链式效应
- Complete-linkage: 倾向于产生紧凑的簇
- Average-linkage: 两者的折中
```

图 6.6: 代码输出结果

聚类方法	ARI	Silhouette	性能评价
Single-linkage	-0.0001	0.1418	很差
Complete-linkage	0.7481	0.5312	良好
Average-linkage	0.8332	0.5444	优秀

表 3: 三种聚类方法结果对比

结论 (make_blobs 数据集):

- 在当前数据集上, **average-linkage** 表现最好
- **Single-linkage**: 容易产生链式效应
- **Complete-linkage**: 倾向于产生紧凑的簇
- **Average-linkage**: 两者的折中

6.2.1 对噪声/离群点的敏感度分析**(1)Single-linkage 的敏感度: 最敏感****原因分析:**

- 基于最小距离, 单个离群点可能连接两个本应分离的簇
- 产生”链式效应”, 对噪声极度敏感
- 在球形数据中表现最差

(2)Complete-linkage 的敏感度: 中等敏感**原因分析:**

- 基于最大距离, 对离群点也敏感但表现不同
- 单个离群点可能阻止本应合并的簇合并
- 倾向于产生过度保守的聚类

(3)Average-linkage 的敏感度: 最不敏感**原因分析:**

- 基于平均距离, 单个离群点的影响被稀释
- 所有样本点平等贡献, 减少极端值的影响
- 在存在噪声的数据中表现最稳定

6.2.2 簇的形状偏好分析**(1)Single-linkage 的簇形状偏好**

适合形状: 不规则形状、链状、非球形簇

原因:

- 最小距离策略允许通过”桥梁点”连接不同区域
- 能够发现任意形状的聚类
- 适合 `make_moons` 类型的非凸数据

(2) Complete-linkage 的簇形状偏好

适合形状: 紧凑球形簇

原因:

- 最大距离策略要求簇内所有点相互接近
- 产生紧密、球形的聚类结构
- 适合 `make_blobs` 类型的球形数据

(3) Average-linkage 的簇形状偏好

适合形状: 通用形状, 中等紧凑

原因:

- 平衡策略, 不极端偏向某种形状
- 既能处理球形簇, 也能适应一定程度的不规则性
- 是最通用的链接方法

6.2.3 可视化结果对比分析

(1) Single-linkage 聚类问题

- **主要问题:** 大部分样本被归为一个簇
- **原因:** 链式效应导致过度合并
- **适用场景:** 仅适合明显链状结构的数据

(2) Complete-linkage 聚类优势

- **优点:** 产生清晰的三个簇
- **不足:** 边界划分可能过于严格
- **适用性:** 球形分离良好的数据

(3) Average-linkage 聚类优势

- **最佳表现:** 簇划分最接近真实分布
- **边界处理:** 自然合理的边界划分
- **通用性:** 在各种数据形状中表现稳定

7 拓展任务：变换聚类簇数 k 分析

7.1 运行结果概览

[步骤5] 拓展要求：测试不同聚类数k (make_blobs 数据集)

[不同聚类数k的性能测试]

k=2:
single : ARI=0.0000, Silhouette=0.3928
complete : ARI=0.5229, Silhouette=0.4568
average : ARI=0.5584, Silhouette=0.4593

k=3:
single : ARI=-0.0001, Silhouette=0.1418
complete : ARI=0.7481, Silhouette=0.5312
average : ARI=0.8332, Silhouette=0.5444

k=4:
single : ARI=0.0000, Silhouette=-0.0537
complete : ARI=0.6273, Silhouette=0.4166
average : ARI=0.8246, Silhouette=0.5290

k=5:
single : ARI=0.0012, Silhouette=-0.2742
complete : ARI=0.6188, Silhouette=0.4060
average : ARI=0.8143, Silhouette=0.4577

k=6:
single : ARI=0.0008, Silhouette=-0.3719
complete : ARI=0.5086, Silhouette=0.3142
average : ARI=0.8164, Silhouette=0.3588

性能对比图已保存

图 7.7: 代码输出结果

k 值	方法	ARI	Silhouette
2	single	0.0000	0.3928
	complete	0.5229	0.4568
	average	0.5584	0.4593
3	single	-0.0001	0.1418
	complete	0.7481	0.5312
	average	0.8332	0.5444
4	single	0.0000	-0.0537
	complete	0.6273	0.4166
	average	0.8246	0.5290
5	single	0.0012	-0.2742
	complete	0.6188	0.4060
	average	0.8143	0.4577
6	single	0.0008	-0.3719
	complete	0.5086	0.3142
	average	0.8164	0.3588

表 4: 不同聚类数 k 的性能测试 (make_blobs 数据集)

7.2 不同 k 值下的图表分析

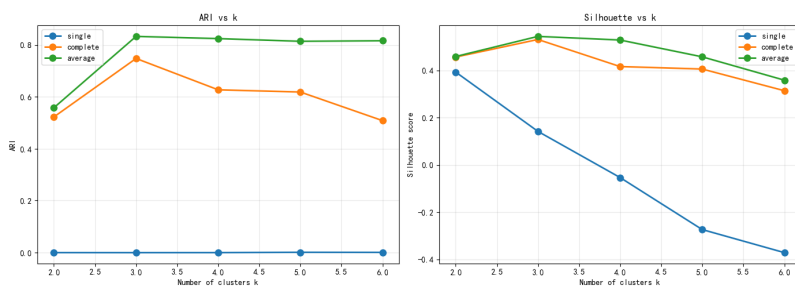


图 7.8: 不同 k 值下的对比图表

7.2.1 ARI vs k 图分析

观察到的趋势:

1. **Average-linkage (绿色):** 在 $k=3$ 时达到峰值 (0.8332), 随后保持高位稳定
2. **Complete-linkage (橙色):** 在 $k=3$ 时达到峰值 (0.7481), 之后明显下降
3. **Single-linkage (蓝色):** 始终接近 0, 几乎没有变化

发现:

- 所有方法的 ARI 在 $k=3$ 时达到最佳, 这与真实簇数一致
- Average-linkage 在不同 k 值下表现最稳定
- Single-linkage 对 k 值变化不敏感 (因为表现一直很差)

7.2.2 Silhouette vs k 图分析

观察到的趋势:

1. **Average-linkage:** $k=3$ 时最高 (0.5444), 之后缓慢下降
2. **Complete-linkage:** $k=3$ 时最高 (0.5312), 之后下降较快
3. **Single-linkage:** 从 $k=2$ 的正值 (0.3928) 快速下降到负值

发现:

- Silhouette 系数能有效反映聚类质量
- 负的 Silhouette 表示样本可能被分配到错误的簇
- Average-linkage 在不同 k 值下保持较高的轮廓系数

7.3 k 值对聚类性能的影响分析

7.3.1 k=2 (小于真实簇数)

现象:

- 所有方法的 ARI 都低于 k=3 时
- 算法被迫将 3 个真实簇合并为 2 个聚类
- Average-linkage 表现最好 (ARI=0.5584)

原因分析:

- 当 k 小于真实簇数时, 不同簇的样本会被强制合并
- Average-linkage 因其平衡性, 合并决策相对合理
- 但 ARI 下降明显, 说明这种合并损失了信息

7.3.2 k=3 (等于真实簇数)

现象:

- 所有方法的性能都达到峰值
- 这与数据的真实结构最匹配
- Average-linkage 表现最佳 (ARI=0.8332)

原因分析:

- 聚类数量与数据真实结构一致
- 算法能准确发现自然存在的簇
- 验证了 k 值选择的重要性

7.3.3 k=4-6 (大于真实簇数)

现象:

- Average-linkage 的 ARI 保持高位 (0.81+)
- Complete-linkage 的 ARI 明显下降
- Single-linkage 的 ARI 始终接近 0

原因分析:

- Average-linkage 能合理地将一个簇细分为多个子簇
- Complete-linkage 倾向于产生紧凑簇, 过度分割会降低质量
- Single-linkage 无法形成有意义的聚类结构

8 github 仓库

代码详见: <https://github.com/Wangxuueuuu/machine-learning/tree/main>